

```
from google.colab import drive
drive.mount('/content/drive')
```

Mounted at /content/drive

```
!git clone https://github.com/indobenchmark/indonlu.git
%cd indonlu
```

Cloning into 'indonlu'...

```
remote: Enumerating objects: 509, done.
remote: Counting objects: 100% (193/193), done.
remote: Compressing objects: 100% (83/83), done.
remote: Total 509 (delta 119), reused 139 (delta 110), pack-reused 316 (from 1)
Receiving objects: 100% (509/509), 9.46 MiB | 13.96 MiB/s, done.
Resolving deltas: 100% (239/239), done.
/content/indonlu
```

```
!pip install google-play-scraper
!pip install pandas
!pip install numpy
!pip install torch
!pip install scikit-learn
!pip install Sastrawi
!pip install nltk
!pip install transformers
```

Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.11/dist-packages (from requests->transformers) (2.4.0) 

Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.11/dist-packages (from requests->transformers) (2025.6.15)

```
import pandas as pd
import numpy as np
import re
from sklearn.model_selection import train_test_split
from sklearn.metrics import confusion_matrix
from sklearn.metrics import classification_report
from collections import defaultdict

# NLP
import nltk
nltk.download('punkt')
# from Sastrawi.Stemmer.StemmerFactory import StemmerFactory
from nltk.probability import FreqDist
from nltk.tokenize import word_tokenize

# Viz
import matplotlib.pyplot as plt
import seaborn as sns
import matplotlib as mpl
from wordcloud import WordCloud

#Model IndoBERT
import random
import torch
import torch.nn.functional as F
from torch import optim
from tqdm import tqdm

from transformers import BertForSequenceClassification, BertConfig, BertTokenizer
from indonlu.utils.data_utils import DocumentSentimentDataset, DocumentSentimentDataLoader
from indonlu.utils.forward_fn import forward_sequence_classification
from indonlu.utils.metrics import document_sentiment_metrics_fn

[🔄] [nltk_data] Downloading package punkt to /root/nltk_data...
[nltk_data] Unzipping tokenizers/punkt.zip.

import multiprocessing
print(multiprocessing.cpu_count())
```

[🔄] 2

✓ Finetuning IndoBERT

```
import random
import numpy as np
import torch

def set_seed(seed):
    random.seed(seed)
    np.random.seed(seed)
    torch.manual_seed(seed)
    torch.cuda.manual_seed(seed)

def count_param(module, trainable=False):
    if trainable:
        return sum(p.numel() for p in module.parameters() if p.requires_grad)
    else:
        return sum(p.numel() for p in module.parameters())

def get_lr(optimizer):
    for param_group in optimizer.param_groups:
        return param_group['lr']

def metrics_to_string(metric_dict):
    string_list = []
    for key, value in metric_dict.items():
        string_list.append('{:}.{:2f}'.format(key, value))
    return ' '.join(string_list)
```

```
# Set random seed
set_seed(27)
```

▼ Load Model

```
from indonlu.utils.data_utils import DocumentSentimentDataset, DocumentSentimentDataLoader
from indonlu.utils.forward_fn import forward_sequence_classification
from indonlu.utils.metrics import document_sentiment_metrics_fn
```

```
from transformers import BertTokenizer, BertConfig, BertForSequenceClassification
```

```
# Load Tokenizer and Config
tokenizer = BertTokenizer.from_pretrained('indobenchmark/indobert-base-p1')
config = BertConfig.from_pretrained('indobenchmark/indobert-base-p1')
config.num_labels = DocumentSentimentDataset.NUM_LABELS
```

```
# Instantiate model
model = BertForSequenceClassification.from_pretrained('indobenchmark/indobert-base-p1', config=config)
```

→ /usr/local/lib/python3.11/dist-packages/huggingface_hub/utils/_auth.py:94: UserWarning:
The secret `HF\_TOKEN` does not exist in your Colab secrets.
To authenticate with the Hugging Face Hub, create a token in your settings tab (<https://huggingface.co/settings/tokens>), set it as secret in your Colab secrets.
You will be able to reuse this secret in all of your notebooks.
Please note that authentication is recommended but still optional to access public models or datasets.

```
warnings.warn(
tokenizer_config.json: 100%                2.00/2.00 [00:00<00:00, 88.3B/s]
vocab.txt: 100%                          229k/229k [00:00<00:00, 9.61MB/s]
special_tokens_map.json: 100%              112/112 [00:00<00:00, 7.49kB/s]
config.json: 100%                        1.53k/1.53k [00:00<00:00, 103kB/s]
pytorch_model.bin: 100%                  498M/498M [00:01<00:00, 211MB/s]
```

Some weights of BertForSequenceClassification were not initialized from the model checkpoint at indobenchmark/indobert-base-p1 and are newly initialized from the normal distribution. You should probably TRAIN this model on a down-stream task to be able to use it for predictions and inference.

```
# Struktur model
model
```

→ BertForSequenceClassification(
 (bert): BertModel(
 (embeddings): BertEmbeddings(
 (word_embeddings): Embedding(50000, 768, padding_idx=0)
 (position_embeddings): Embedding(512, 768)
 (token_type_embeddings): Embedding(2, 768)
 (LayerNorm): LayerNorm((768,), eps=1e-12, elementwise_affine=True)
 (dropout): Dropout(p=0.1, inplace=False)
)
 (encoder): BertEncoder(
 (layer): ModuleList(
 (0-11): 12 x BertLayer(
 (attention): BertAttention(
 (self): BertSdpaSelfAttention(
 (query): Linear(in_features=768, out_features=768, bias=True)
 (key): Linear(in_features=768, out_features=768, bias=True)
 (value): Linear(in_features=768, out_features=768, bias=True)
 (dropout): Dropout(p=0.1, inplace=False)
)
 (output): BertSelfOutput(
 (dense): Linear(in_features=768, out_features=768, bias=True)
 (LayerNorm): LayerNorm((768,), eps=1e-12, elementwise_affine=True)
 (dropout): Dropout(p=0.1, inplace=False)
)
)
 (intermediate): BertIntermediate(
 (dense): Linear(in_features=768, out_features=3072, bias=True)
 (intermediate_act_fn): GELUActivation()
)
 (output): BertOutput(
 (dense): Linear(in_features=3072, out_features=768, bias=True)
 (LayerNorm): LayerNorm((768,), eps=1e-12, elementwise_affine=True)
 (dropout): Dropout(p=0.1, inplace=False)
)
)
)
)
)
)

```

    )
    (pooler): BertPooler(
      (dense): Linear(in_features=768, out_features=768, bias=True)
      (activation): Tanh()
    )
  )
  (dropout): Dropout(p=0.1, inplace=False)
  (classifier): Linear(in_features=768, out_features=3, bias=True)
)

```

✓ Prepare Dataset

```

train_dataset_path = '/content/drive/MyDrive/Colab Notebooks/SKRIPSI/wondr_ds_train_70.tsv'
valid_dataset_path = '/content/drive/MyDrive/Colab Notebooks/SKRIPSI/wondr_ds_validation_15.tsv'
test_dataset_path = '/content/drive/MyDrive/Colab Notebooks/SKRIPSI/wondr_ds_test_15.tsv'

```

```

train_dataset_path = '/content/drive/MyDrive/Colab Notebooks/SKRIPSI/wondr_ds_train_80.tsv'
valid_dataset_path = '/content/drive/MyDrive/Colab Notebooks/SKRIPSI/wondr_ds_validation_10.tsv'
test_dataset_path = '/content/drive/MyDrive/Colab Notebooks/SKRIPSI/wondr_ds_test_10.tsv'

```

```

# fungsi dataset loader dari utils IndoNLU
train_dataset = DocumentSentimentDataset(train_dataset_path, tokenizer, lowercase=True)
valid_dataset = DocumentSentimentDataset(valid_dataset_path, tokenizer, lowercase=True)
test_dataset = DocumentSentimentDataset(test_dataset_path, tokenizer, lowercase=True)

train_loader = DocumentSentimentDataLoader(dataset=train_dataset, max_seq_len=512, batch_size=16, num_workers=2, shuffle=True)
valid_loader = DocumentSentimentDataLoader(dataset=valid_dataset, max_seq_len=512, batch_size=16, num_workers=2, shuffle=False)
test_loader = DocumentSentimentDataLoader(dataset=test_dataset, max_seq_len=512, batch_size=16, num_workers=2, shuffle=False)

```

```

w2i, i2w = DocumentSentimentDataset.LABEL2INDEX, DocumentSentimentDataset.INDEX2LABEL
print(w2i) #word to index
print(i2w) #index to word

```

```

↔ {'positive': 0, 'neutral': 1, 'negative': 2}
   {0: 'positive', 1: 'neutral', 2: 'negative'}

```

✓ Uji coba pre-trained model

```

import torch.nn.functional as F

text = 'tidak bisa transfer antar bank'
subwords = tokenizer.encode(text)
subwords = torch.LongTensor(subwords).view(1, -1).to(model.device)

logits = model(subwords)[0]
label = torch.topk(logits, k=1, dim=-1)[1].squeeze().item()

print(f'Text: {text} | Label : {i2w[label]} ({F.softmax(logits, dim=-1).squeeze()[label] * 100:.3f}%)')

↔ Text: tidak bisa transfer antar bank | Label : neutral (55.560%)

```

✓ Fine Tuning & Prediksi Evaluation

```

# Tentukan optimizer
optimizer = optim.Adam(model.parameters(), lr=3e-6)
model = model.cuda()

```

```

# Train
n_epochs = 15
history = defaultdict(list)

```

```

for epoch in range(n_epochs):
    model.train()
    torch.set_grad_enabled(True)

```

```

    total_train_loss = 0
    list_hus_train, list_label = [], []

```

```
list_hyp_train, list_label_train = [], []
```

```
train_pbar = tqdm(train_loader, leave=True, total=len(train_loader))
for i, batch_data in enumerate(train_pbar):
    loss, batch_hyp, batch_label = forward_sequence_classification(model, batch_data[:-1], i2w=i2w, device='cuda')

    optimizer.zero_grad()
    loss.backward()
    optimizer.step()

    tr_loss = loss.item()
    total_train_loss += tr_loss

    list_hyp_train += batch_hyp
    list_label_train += batch_label

    train_pbar.set_description(f"(Epoch {epoch+1}) TRAINING...")

train_metrics = document_sentiment_metrics_fn(list_hyp_train, list_label_train)

print(f"\nEpoch {epoch+1} - Training Accuracy: {train_metrics['ACC']:.4f}")

history['train_acc'].append(train_metrics['ACC'])

# Validation
model.eval()
torch.set_grad_enabled(False)

list_hyp_valid, list_label_valid = [], []

valid_pbar = tqdm(valid_loader, leave=True, total=len(valid_loader))
for i, batch_data in enumerate(valid_pbar):
    _, batch_hyp, batch_label = forward_sequence_classification(model, batch_data[:-1], i2w=i2w, device='cuda')
    list_hyp_valid += batch_hyp
    list_label_valid += batch_label

    valid_pbar.set_description("Validating...")

valid_metrics = document_sentiment_metrics_fn(list_hyp_valid, list_label_valid)

print(f"Epoch {epoch+1} - Validation Accuracy: {valid_metrics['ACC']:.4f}")
print("=====\n")

history['val_acc'].append(valid_metrics['ACC'])
```



```

=====
(Epoch 13) TRAINING...: 100%|██████████| 438/438 [02:58<00:00, 2.45it/s]

Epoch 13 - Training Accuracy: 0.9726
Validating...: 100%|██████████| 94/94 [00:13<00:00, 7.01it/s]
Epoch 13 - Validation Accuracy: 0.8327
=====

(Epoch 14) TRAINING...: 100%|██████████| 438/438 [02:59<00:00, 2.43it/s]

Epoch 14 - Training Accuracy: 0.9773
Validating...: 100%|██████████| 94/94 [00:13<00:00, 7.00it/s]
Epoch 14 - Validation Accuracy: 0.8493
=====

(Epoch 15) TRAINING...: 100%|██████████| 438/438 [02:59<00:00, 2.44it/s]

Epoch 15 - Training Accuracy: 0.9819
Validating...: 100%|██████████| 94/94 [00:13<00:00, 7.03it/s]Epoch 15 - Validation Accuracy: 0.8727
=====

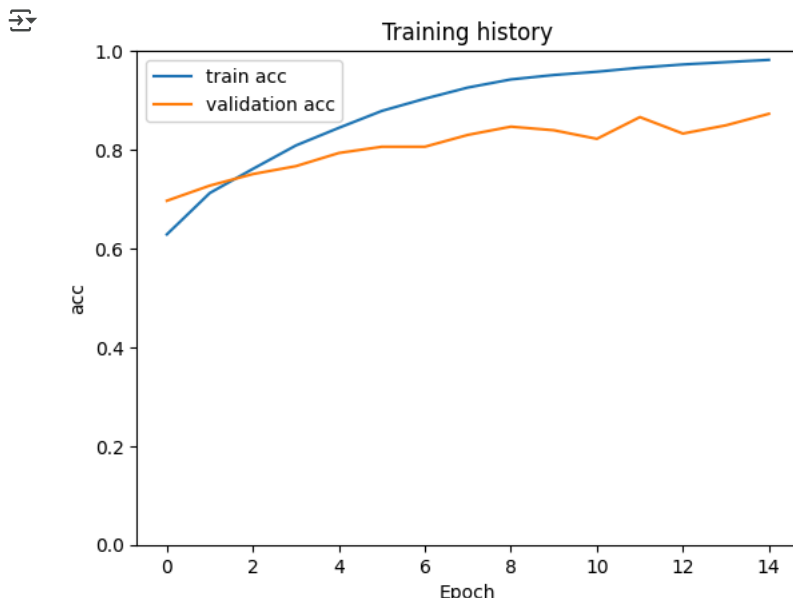
```

✓ Learning Curve

```

plt.plot(history['train_acc'], label='train acc')
plt.plot(history['val_acc'], label='validation acc')
plt.title('Training history')
plt.ylabel('acc')
plt.xlabel('Epoch')
plt.legend()
plt.ylim([0, 1]);

```



```

# Simpan Hasil Prediksi Validation Set
val_df = pd.read_csv(valid_dataset_path, sep='\t', names=['review_text', 'category'])
val_df['pred'] = list_hyp_valid
val_df.head()
val_df.to_csv('/content/drive/MyDrive/Colab Notebooks/SKRIPSI/wondr_val_result_70_30_16_3e5.csv', index=False)

```

```

# Simpan Hasil Prediksi Validation Set
val_df = pd.read_csv(valid_dataset_path, sep='\t', names=['review_text', 'category'])
val_df['pred'] = list_hyp_valid
val_df.head()
val_df.to_csv('/content/drive/MyDrive/Colab Notebooks/SKRIPSI/wondr_val_result_70_30_16_3e6.csv', index=False)

```

```

# Simpan Hasil Prediksi Validation Set
val_df = pd.read_csv(valid_dataset_path, sep='\t', names=['review_text', 'category'])
val_df['pred'] = list_hyp_valid

```

```

val_df.head()
val_df.to_csv('/content/drive/MyDrive/Colab Notebooks/SKRIPSI/wondr_val_result_70_30_32_3e5.csv', index=False)

# Simpan Hasil Prediksi Validation Set
val_df = pd.read_csv(valid_dataset_path, sep='\t', names=['review_text', 'category'])
val_df['pred'] = list_hyp_valid
val_df.head()
val_df.to_csv('/content/drive/MyDrive/Colab Notebooks/SKRIPSI/wondr_val_result_70_30_32_3e6.csv', index=False)

# Simpan Hasil Prediksi Validation Set
val_df = pd.read_csv(valid_dataset_path, sep='\t', names=['review_text', 'category'])
val_df['pred'] = list_hyp_valid
val_df.head()
val_df.to_csv('/content/drive/MyDrive/Colab Notebooks/SKRIPSI/wondr_val_result_80_20_16_3e5.csv', index=False)

# Simpan Hasil Prediksi Validation Set
val_df = pd.read_csv(valid_dataset_path, sep='\t', names=['review_text', 'category'])
val_df['pred'] = list_hyp_valid
val_df.head()
val_df.to_csv('/content/drive/MyDrive/Colab Notebooks/SKRIPSI/wondr_val_result_80_20_16_3e6.csv', index=False)

# Simpan Hasil Prediksi Validation Set
val_df = pd.read_csv(valid_dataset_path, sep='\t', names=['review_text', 'category'])
val_df['pred'] = list_hyp_valid
val_df.head()
val_df.to_csv('/content/drive/MyDrive/Colab Notebooks/SKRIPSI/wondr_val_result_80_20_32_3e5.csv', index=False)

# Simpan Hasil Prediksi Validation Set
val_df = pd.read_csv(valid_dataset_path, sep='\t', names=['review_text', 'category'])
val_df['pred'] = list_hyp_valid
val_df.head()
val_df.to_csv('/content/drive/MyDrive/Colab Notebooks/SKRIPSI/wondr_val_result_80_20_32_3e6.csv', index=False)

```

✓ Prediksi Test Set

```

# Prediksi test set
model.eval()
torch.set_grad_enabled(False)

total_loss, total_correct, total_labels = 0, 0, 0
pred, list_label = [], []

pbar = tqdm(test_loader, leave=True, total=len(test_loader))
for i, batch_data in enumerate(pbar):
    _, batch_hyp, _ = forward_sequence_classification(model, batch_data[:-1], i2w=i2w, device='cuda')
    pred += batch_hyp

🔄 100%|██████████| 94/94 [00:12<00:00, 7.70it/s]

# Simpan prediksi test set
test_df = pd.read_csv(test_dataset_path, sep='\t', names=['review_text', 'category'])
test_df['pred'] = pred
test_df.head()
test_df.to_csv('/content/drive/MyDrive/Colab Notebooks/SKRIPSI/wondr_test_result_70_30_16_3e5.csv', index=False)

# Simpan prediksi test set
test_df = pd.read_csv(test_dataset_path, sep='\t', names=['review_text', 'category'])
test_df['pred'] = pred
test_df.head()
test_df.to_csv('/content/drive/MyDrive/Colab Notebooks/SKRIPSI/wondr_test_result_70_30_16_3e6.csv', index=False)

# Simpan prediksi test set
test_df = pd.read_csv(test_dataset_path, sep='\t', names=['review_text', 'category'])
test_df['pred'] = pred
test_df.head()
test_df.to_csv('/content/drive/MyDrive/Colab Notebooks/SKRIPSI/wondr_test_result_70_30_32_3e5.csv', index=False)

```

```
# Simpan prediksi test set
test_df = pd.read_csv(test_dataset_path, sep='\t', names=['review_text', 'category'])
test_df['pred'] = pred
test_df.head()
test_df.to_csv('/content/drive/MyDrive/Colab Notebooks/SKRIPSI/wondr_test_result_70_30_32_3e6.csv', index=False)

# Simpan prediksi test set
test_df = pd.read_csv(test_dataset_path, sep='\t', names=['review_text', 'category'])
test_df['pred'] = pred
test_df.head()
test_df.to_csv('/content/drive/MyDrive/Colab Notebooks/SKRIPSI/wondr_test_result_80_20_16_3e5.csv', index=False)

# Simpan prediksi test set
test_df = pd.read_csv(test_dataset_path, sep='\t', names=['review_text', 'category'])
test_df['pred'] = pred
test_df.head()
test_df.to_csv('/content/drive/MyDrive/Colab Notebooks/SKRIPSI/wondr_test_result_80_20_16_3e6.csv', index=False)

# Simpan prediksi test set
test_df = pd.read_csv(test_dataset_path, sep='\t', names=['review_text', 'category'])
test_df['pred'] = pred
test_df.head()
test_df.to_csv('/content/drive/MyDrive/Colab Notebooks/SKRIPSI/wondr_test_result_80_20_32_3e5.csv', index=False)

# Simpan prediksi test set
test_df = pd.read_csv(test_dataset_path, sep='\t', names=['review_text', 'category'])
test_df['pred'] = pred
test_df.head()
test_df.to_csv('/content/drive/MyDrive/Colab Notebooks/SKRIPSI/wondr_test_result_80_20_32_3e6.csv', index=False)
```

✓ Test fine-tuned model on sample sentences

```
text = 'transfer antar bank lancar'
subwords = tokenizer.encode(text)
subwords = torch.LongTensor(subwords).view(1, -1).to(model.device)

logits = model(subwords)[0]
label = torch.topk(logits, k=1, dim=-1)[1].squeeze().item()

print(f'Text: {text} | Label : {i2w[label]} ({F.softmax(logits, dim=-1).squeeze()[label] * 100:.3f}%)')

↩ Text: transfer antar bank lancar | Label : positive (85.633%)
```

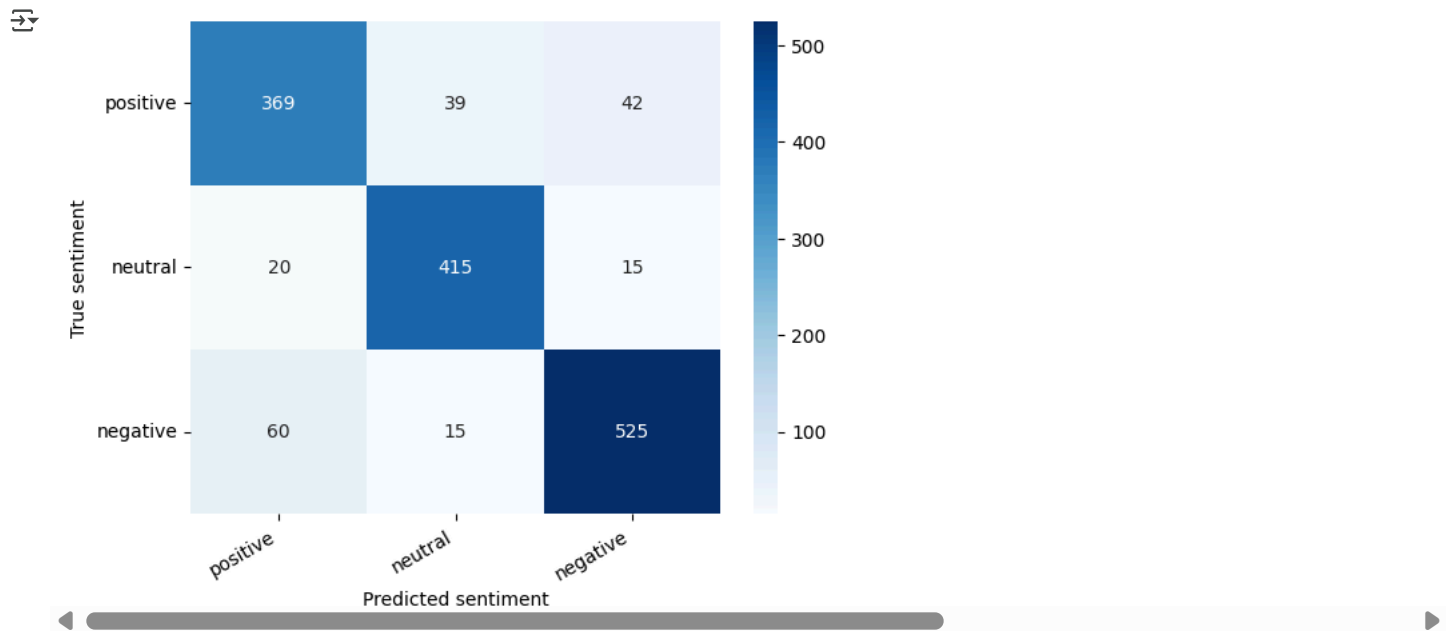
✓ Evaluasi

```
val_real = val_df.category
val_pred = val_df.pred

test_real = test_df.category
test_pred = test_df.pred

def show_confusion_matrix(confusion_matrix):
    hmap = sns.heatmap(confusion_matrix, annot=True, fmt="d", cmap="Blues")
    hmap.yaxis.set_ticklabels(hmap.yaxis.get_ticklabels(), rotation=0, ha='right')
    hmap.xaxis.set_ticklabels(hmap.xaxis.get_ticklabels(), rotation=30, ha='right')
    plt.ylabel('True sentiment')
    plt.xlabel('Predicted sentiment');

cm = confusion_matrix(val_real, val_pred)
df_cm = pd.DataFrame(cm, index=['positive', 'neutral', 'negative'], columns=['positive', 'neutral', 'negative'])
show_confusion_matrix(df_cm)
```

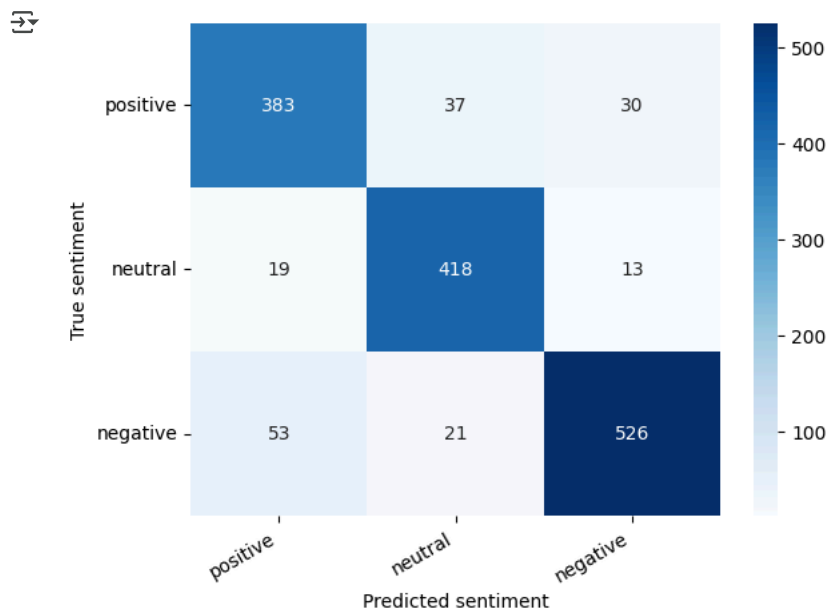


```
print(classification_report(val_real, val_pred, target_names=['positive', 'neutral', 'negative']))
```

	precision	recall	f1-score	support
positive	0.82	0.82	0.82	450
neutral	0.88	0.92	0.90	450
negative	0.90	0.88	0.89	600
accuracy			0.87	1500
macro avg	0.87	0.87	0.87	1500
weighted avg	0.87	0.87	0.87	1500

```
def show_confusion_matrix(confusion_matrix):
    hmap = sns.heatmap(confusion_matrix, annot=True, fmt="d", cmap="Blues")
    hmap.yaxis.set_ticklabels(hmap.yaxis.get_ticklabels(), rotation=0, ha='right')
    hmap.xaxis.set_ticklabels(hmap.xaxis.get_ticklabels(), rotation=30, ha='right')
    plt.ylabel('True sentiment')
    plt.xlabel('Predicted sentiment');

cm = confusion_matrix(test_real, test_pred)
df_cm = pd.DataFrame(cm, index=['positive', 'neutral', 'negative'], columns=['positive', 'neutral', 'negative'])
show_confusion_matrix(df_cm)
```



```
print(classification_report(test_real, test_pred, target_names=['positive', 'neutral', 'negative']))
```



	precision	recall	f1-score	support
positive	0.84	0.85	0.85	450
neutral	0.88	0.93	0.90	450
negative	0.92	0.88	0.90	600
accuracy			0.88	1500
macro avg	0.88	0.89	0.88	1500
weighted avg	0.89	0.88	0.88	1500